Introduction

Predictive Modeling Using Machine Learning

This project focuses on building a machine learning model to predict student outcomes using a Student Performance dataset. Random Forest Classification is applied to predict whether a student will pass or fail based on academic and demographic features.

The model is trained and tested using supervised learning techniques, and performance is evaluated using accuracy score, confusion matrix, and ROC curve visualization.

Upload Dataset

```
from google.colab import files

uploaded = files.upload()
```

```
Choose Files Student_Pe…Dataset.csv
Student_Performance_Dataset.csv(text/csv) - 363184 bytes, last modified: 3/12/2026 - 100% done
Saving Student_Performance_Dataset.csv to Student_Performance_Dataset.csv
```

Import Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)
```

Load Dataset

```
df = pd.read_csv(
    "Student_Performance_Dataset.csv"
)
```

Explore Dataset

```
df.head()
```

	Student_ID	Age	Gender	Class	Study_Hours_Per_Day	Attendance_Percentage
0	S0001	15	Male	12	1.0	65
1	S0002	19	Female	9	1.6	58
2	S0003	14	Female	12	3.6	64
3	S0004	18	Female	9	5.5	68
4	S0005	14	Male	10	5.0	80

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 16 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Student_ID                 5000 non-null   object
 1   Age                        5000 non-null   int64
 2   Gender                     5000 non-null   object
 3   Class                      5000 non-null   int64
 4   Study_Hours_Per_Day        5000 non-null   float64
 5   Attendance_Percentage      5000 non-null   int64
 6   Parental_Education         5000 non-null   object
 7   Internet_Access            5000 non-null   object
 8   Extracurricular_Activities 5000 non-null   object
 9   Math_Score                 5000 non-null   int64
 10  Science_Score              5000 non-null   int64
 11  English_Score              5000 non-null   int64
 12  Previous_Year_Score        5000 non-null   int64
 13  Final_Percentage           5000 non-null   float64
 14  Performance_Level          5000 non-null   object
 15  Pass_Fail                  5000 non-null   object
dtypes: float64(2), int64(7), object(7)
memory usage: 625.1+ KB
```

```
df.describe()
```

	Age	Class	Study_Hours_Per_Day	Attendance_Percentage	5
count	5000.000000	5000.000000	5000.000000	5000.000000	
mean	16.508800	10.496400	3.287260	74.919800	
std	1.718637	1.106812	1.587979	14.673842	
min	14.000000	9.000000	0.500000	50.000000	
25%	15.000000	10.000000	1.900000	62.000000	
50%	17.000000	10.000000	3.300000	75.000000	
75%	18.000000	11.000000	4.700000	88.000000	
max	19.000000	12.000000	6.000000	100.000000	

Check Missing Value

```
df.isnull().sum()
```

	0
Student_ID	0
Age	0
Gender	0
Class	0
Study_Hours_Per_Day	0
Attendance_Percentage	0
Parental_Education	0
Internet_Access	0
Extracurricular_Activities	0
Math_Score	0
Science_Score	0
English_Score	0
Previous_Year_Score	0
Final_Percentage	0
Performance_Level	0
Pass_Fail	0

dtype: int64

Missing Value Analysis

The dataset was checked for missing values using the `isnull().sum()` function.

Result: No missing values were found in the dataset.

Check Duplicate Value

```
df.duplicated().sum()
```

```
np.int64(0)
```

Duplicate Analysis

The dataset was checked for duplicate records.

Result: No duplicate rows were found.

Data Preprocessing

Machine learning models require numeric values.

Convert categorical columns.

```
le = LabelEncoder()

df['Gender'] = le.fit_transform(
    df['Gender']
)

df['Class'] = le.fit_transform(
    df['Class']
)

df['Parental_Education'] = le.fit_transform(
    df['Parental_Education']
)

df['Internet_Access'] = le.fit_transform(
    df['Internet_Access']
)

df['Extracurricular_Activities'] = le.fit_transform(
    df['Extracurricular_Activities']
)

df['Performance_Level'] = le.fit_transform(
    df['Performance_Level']
)
```

```
df['Pass_Fail'] = le.fit_transform(
    df['Pass_Fail']
)
```

Feature Selection

We will predict:

Pass_Fail

```
X = df.drop(
    ['Pass_Fail','Student_ID'],
    axis=1
)

y = df['Pass_Fail']
```

Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Train Random Forest

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(
    X_train,
    y_train
)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Make Predictions

```
y_pred = rf_model.predict(
    X_test
)
```

Evaluate Model Accuracy

```
accuracy = accuracy_score(
    y_test,
    y_pred
)

print(
    "Random Forest Accuracy:",
    accuracy
)
```

```
Random Forest Accuracy: 1.0
```

Classification Report

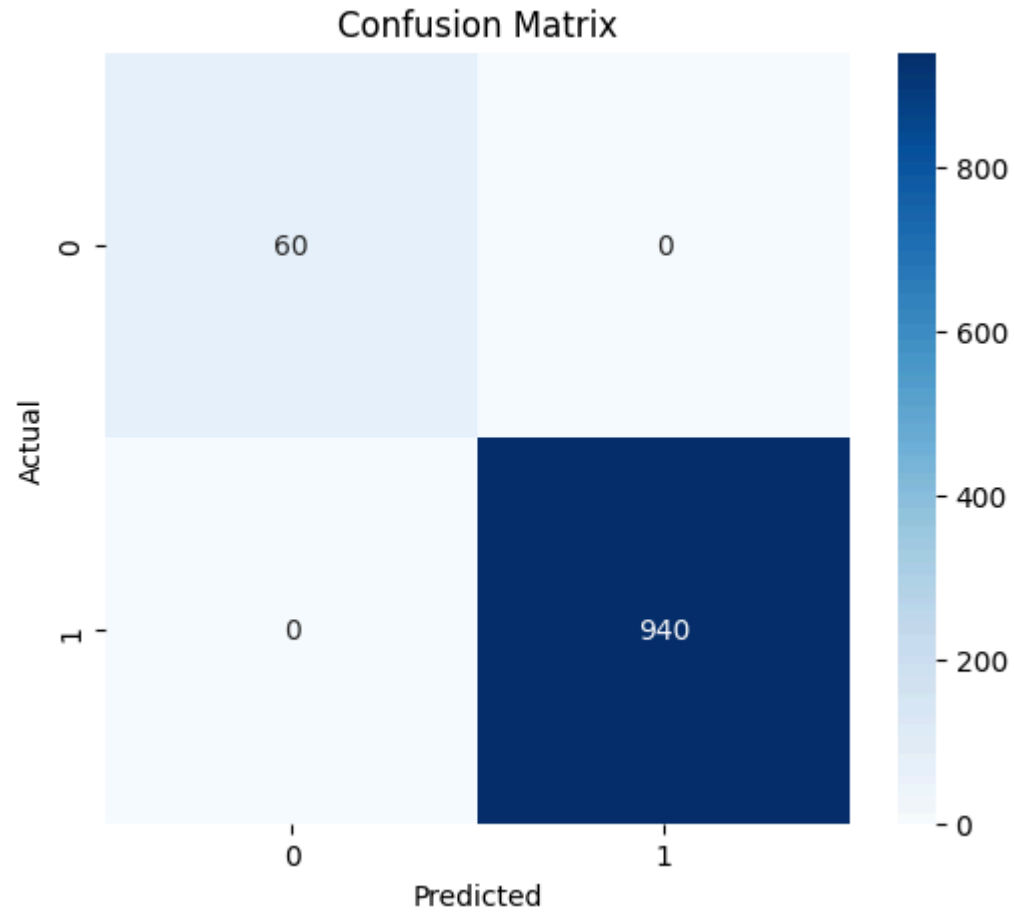
```
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        60
           1       1.00      1.00      1.00       940

    accuracy                           1.00      1000
   macro avg       1.00      1.00      1.00      1000
weighted avg       1.00      1.00      1.00      1000
```

Confusion Matrix Visualization

```
cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title(
    "Confusion Matrix"
)
```

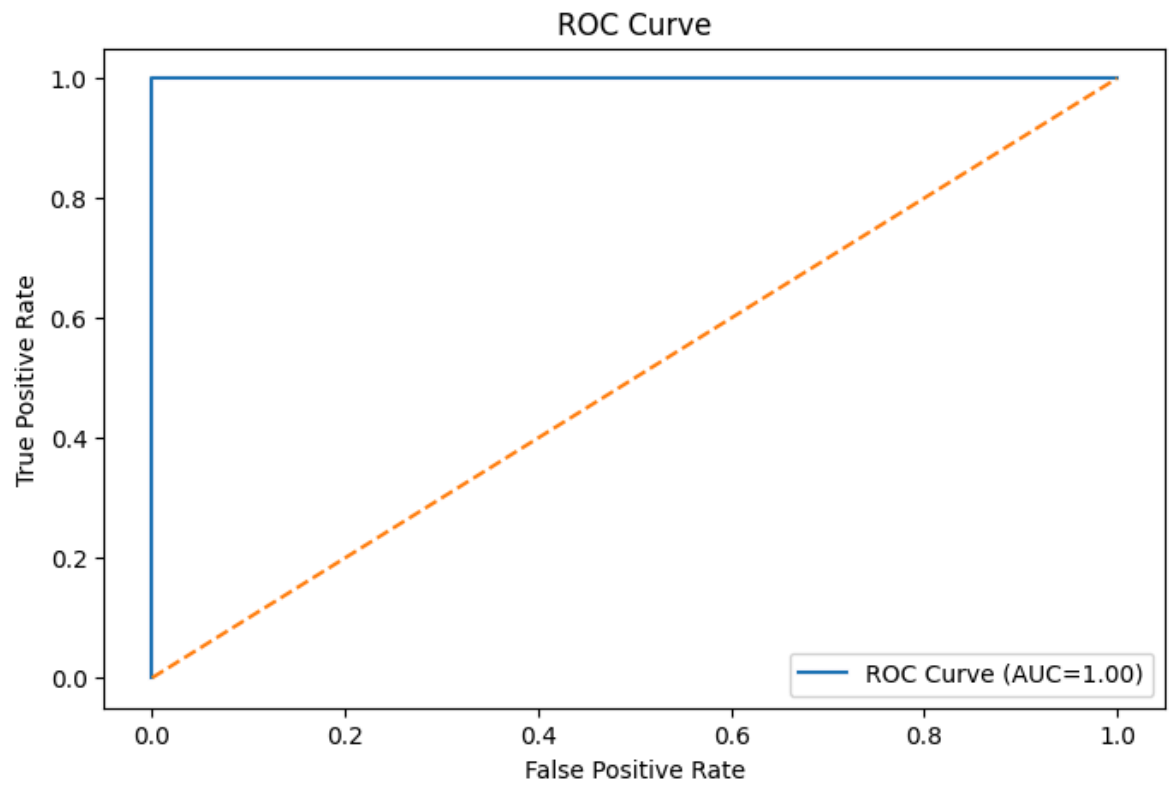
```
plt.xlabel(
    "Predicted"
)

plt.ylabel(
    "Actual"
)

plt.show()
```



ROC Curve Visualization

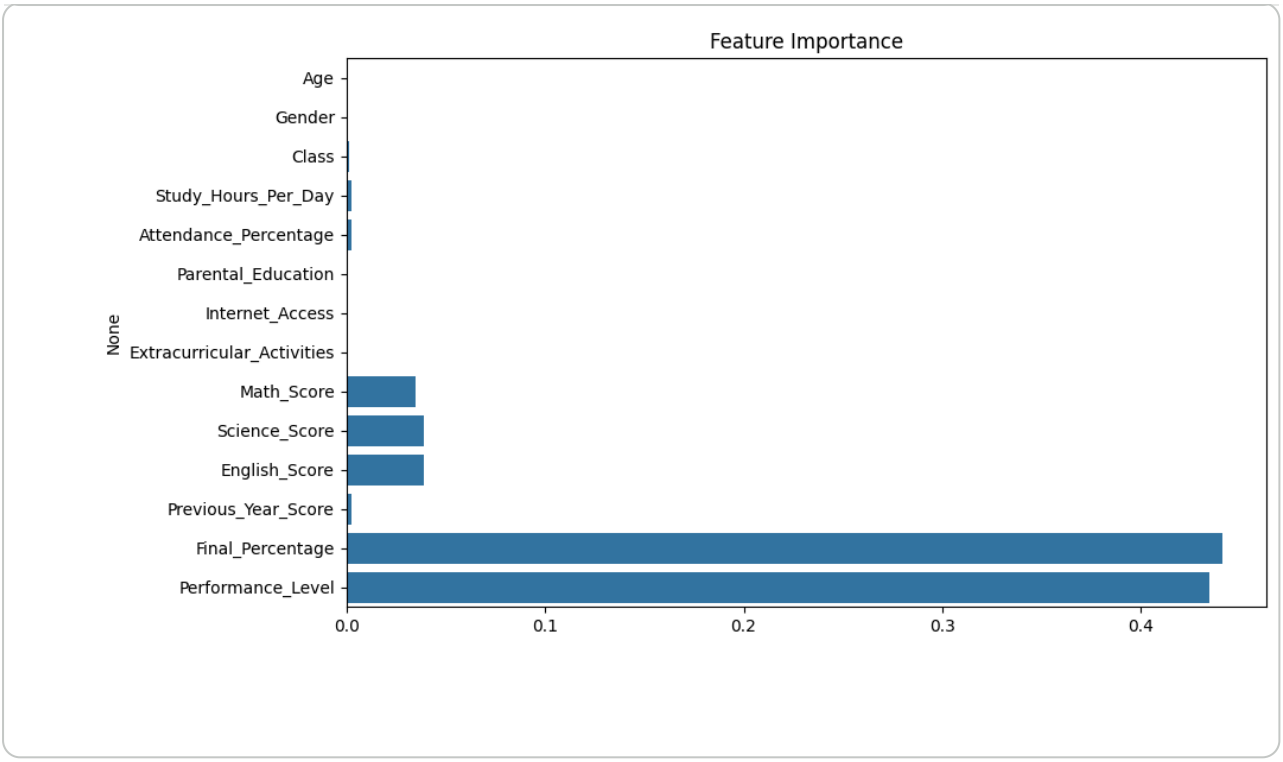
```
y_prob = rf_model.predict_proba(
    X_test
)[:,1]

fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob
)

roc_auc = auc(
    fpr,
    tpr
)
```

```
plt.figure(figsize=(8,5))

plt.plot(
    fpr,
    tpr,
    label='ROC Curve (AUC=%0.2f)' % roc_auc
)

plt.plot(
    [0,1],
    [0,1],
    linestyle='--'
)

plt.xlabel(
    "False Positive Rate"
)

plt.ylabel(
    "True Positive Rate"
)

plt.title(
    "ROC Curve"
)

plt.legend()

plt.show()
```



Feature Importance Visualization

```
importance = rf_model.feature_importances_

feature_names = X.columns

plt.figure(figsize=(10,6))

sns.barplot(
    x=importance,
    y=feature_names
)

plt.title(
    "Feature Importance"
)

plt.show()
```



Close

```
 **Conclusion**

This project demonstrated predictive mode
on a Student Performance dataset.

The model successfully predicted student
demographic features.
```

Conclusion

This project demonstrated predictive modeling using the Random Forest algorithm on a Student Performance dataset.

The model successfully predicted